



```

1  module dual_port_ram#(parameter DATA_W = 16, ADDR_W = 4)
2  (
3  input clk, rst,
4  input [ADDR_W-1:0] addr_a, addr_b,
5  input [DATA_W-1:0] din_a, din_b,
6  input we_a, we_b,
7  output reg collision,
8  output reg [DATA_W-1:0] dout_a, dout_b
9  );
10
11  reg [DATA_W-1:0] mem [0:ADDR_W-1];
12  integer i;
13  always @(posedge clk )
14  begin
15      collision <=0;
16      if(rst)
17      begin
18          for(i = 0; i<ADDR_W; i = i+1) mem[i] <=0;
19          dout_a<=0;
20          dout_b<=0;
21      end
22
23      if(we_a && we_b) // double access
24      begin
25          if(addr_a == addr_b) // collision
26          begin
27              collision <=1;
28              mem[addr_a]<=din_a;
29          end
30          else // no collision
31          begin
32              mem[addr_b]<=din_b;
33              mem[addr_a]<=din_a;
34          end
35      end
36      else if(we_a) mem[addr_a]<=din_a;
37      else if(we_b) mem[addr_b]<=din_b;
38      dout_a<=mem[addr_a];
39      dout_b<=mem[addr_b];
40  end
41  endmodule

```

```

1  module dual_port_ram#(parameter DATA_W = 16, ADDR_W = 4)
2  (
3  input clk, rst,
4  input [ADDR_W-1:0] addr_a, addr_b,
5  input [DATA_W-1:0] din_a, din_b,
6  input we_a, we_b,
7  output reg collision,
8  output reg [DATA_W-1:0] dout_a, dout_b
9  );
10
11 reg [DATA_W-1:0] mem [0:ADDR_W-1];
12 integer i;
13 always @(posedge clk )
14 begin
15     collision <=0;
16     if(rst)
17     begin
18         for(i = 0; i<ADDR_W; i = i+1) mem[i] <=0;
19         dout_a<=0;
20         dout_b<=0;
21     end
22
23     if(we_a && we_b) // double access
24     begin
25         if(addr_a == addr_b) // collision
26         begin
27             collision <=1;
28             mem[addr_a]<=din_a;
29         end
30         else // no collision
31         begin
32             mem[addr_b]<=din_b;
33             mem[addr_a]<=din_a;
34         end
35     end
36     else if(we_a) mem[addr_a]<=din_a;
37     else if(we_b) mem[addr_b]<=din_b;
38     dout_a<=mem[addr_a];
39     dout_b<=mem[addr_b];
40 end
41 endmodule
42

```

The screenshot displays a Verilog simulation environment with two main panes. The left pane shows the testbench code, and the right pane shows the resulting waveform.

Testbench Code (Left Pane):

```

+ /dual_port_ram_tb/clk 0
+ /dual_port_ram_tb/rst 0
+ /dual_port_ram_tb/... 1111
+ /dual_port_ram_tb/... 0000
+ /dual_port_ram_tb/... 1001101010111100
+ /dual_port_ram_tb/... 1101111011110000
+ /dual_port_ram_tb/... 0
+ /dual_port_ram_tb/... 0
+ /dual_port_ram_tb/... St0
+ /dual_port_ram_tb/... xxxxxxxxxxxxxxxxxxxx
+ /dual_port_ram_tb/... 0001001000110100
+ /dual_port_ram_tb/i 1

```

Waveform (Right Pane):

The waveform shows the timing of various signals. A yellow box highlights the signal `sim:/dual_port_ram_tb/we_b @ 5 ps`, which is a 1-bit control signal. The waveform also shows other signals such as `0000`, `0001`, `0010`, `0011`, `0010`, `0100`, `00000000000000...`, `0001001000110100`, `1001101010111100`, `01010111001111000`, `1101111011110000`, `0101011...`, `00010010...`, `1001101...`, `0`, `2`, `-1`, `1`, `3`, and `5`.

```

1  module fifo #(parameter data_w =16 , depth = 64)
2  (
3  input clk, rst_n,
4  input wr_en,rd_en,
5  input [data_w-1:0]din,
6  output reg [data_w-1:0]dout,
7  output full ,empty , almost_full, almost_empty
8  );
9
10 reg [depth:0] w_ptr, r_ptr;
11 reg [data_w-1:0] fifo [0:depth-1];
12
13 always @ ( posedge clk )
14 begin
15 w_ptr <= 0;
16 r_ptr <= 0;
17
18 if(!rst_n)
19 begin
20     w_ptr <= 0;
21     r_ptr <=0;
22     dout <= 0;
23 end
24
25 // write
26 if(wr_en && !full)
27 begin
28     fifo[w_ptr] <=din ;
29     w_ptr <= w_ptr + 1;
30 end
31
32 //read
33 if(rd_en && !empty)
34 begin
35     dout <= fifo[r_ptr];
36     r_ptr <= r_ptr + 1;
37 end
38
39
40 end
41
42 assign empty = (r_ptr == w_ptr);
43 assign full  = (r_ptr[depth] != w_ptr[depth] && r_ptr[depth-1 : 0 ] == w_ptr[depth-1 : 0]);
44 assign almost_empty = (w_ptr[depth-1 : 0] - r_ptr[depth-1 : 0] <= 2  && w_ptr[depth-1 : 0] > r_ptr[depth-1 : 0]);
45 assign almost_full = (r_ptr[depth-1 : 0] - w_ptr[depth-1 : 0] <= 2  && r_ptr[depth-1 : 0] > w_ptr[depth-1 : 0]);
46
47 endmodule

```

```
module ALU (A,B,opcode,clk,rst,result);
```

```
input [1:0] A,B,opcode;
```

```
input clk,rst;
```

```
output [3:0] result;
```

Output Reg

```
always @(posedge clk or posedge rst) begin
```

```
if (!rst) begin
```

neg edge

```
result <= 0;
```

```
end
```

```
else
```

```
case (opcode)
```

```
0: result <= A + B;
```

```
1: result <= A * B;
```

```
2 0: result <= A | B;
```

```
3: result <= A ^ B;
```

```
endcase
```

```
end
```



```
1  module alu (  
2    input [1:0] a,b,opcode,  
3    input clk, rst,  
4    output reg [3:0] res  
5  );  
6  
7  c_addsub_0 your_instance_name (  
8    .A(A), // input wire [1 : 0] A  
9    .B(B), // input wire [1 : 0] B  
10   .S(S) // output wire [2 : 0] S  
11  );  
12  
13  mult_gen_0 your_instance_name (  
14    .CLK(CLK), // input wire CLK  
15    .A(A),      // input wire [1 : 0] A  
16    .B(B),      // input wire [1 : 0] B  
17    .P(P)       // output wire [3 : 0] P  
18  );  
19  
20  
21  always @(posedge clk or negedge rst)  
22  begin  
23  
24    if(!rst) res <= 0;  
25    else  
26    begin  
27      case(opcode)  
28        0: res <= S;  
29        1: res <= P;  
30        2: res <= a|b;  
31        3: res <= a^b;  
32      endcase  
33    end  
34  end  
35  endmodule
```




```
1  module alu_tb;
2
3
4  reg [1:0] a,b,opcode;
5  reg clk, rst;
6  wire [3:0] res;
7
8  alu dut (a,b,opcode,clk,rst,res);
9
10 task valid_output;
11 begin
12     case(opcode)
13     0:if(res != a+b) $display("time=%0t | a= %d | b= %d | opcode= %d | output= %d <<FIALED>> ",$time, a, b, opcode, res);
14     1:if(res != a*b) $display("time=%0t | a= %d | b= %d | opcode= %d | output= %d <<FIALED>> ",$time, a, b, opcode, res);
15     2:if(res != a|b) $display("time=%0t | a= %d | b= %d | opcode= %d | output= %d <<FIALED>> ",$time, a, b, opcode, res);
16     3:if(res != a^b) $display("time=%0t | a= %d | b= %d | opcode= %d | output= %d <<FIALED>> ",$time, a, b, opcode, res);
17     endcase
18 end
19 endtask
20
21 always #5 clk = ~clk;
22
23 initial begin
24     a = 0 ; b = 0 ; opcode = 0; clk = 0,rst=0;
25     #10;
26     rst =1;
27     repeat(15) begin
28         // random values assign
29         a =$random;
30         b =$random;
31         opcode =$random % 4; // Ensure opcode stays within 0-3
32         valid_output(); // transcript screen validation output
33         #10; // wait 10 time unit for the next test
34
35     end // end of repeat
36     $stop;
37 end // end of initial
38
39
40 initial begin
41     $monitor("time=%0t | a= %d | b= %d | opcode= %d | output= %d",$time, a, b, opcode, res);
42 end
43 endmodule
44
45
46
47
48
49
50
```

Flow Navigator

PROJECT MANAGER

IP INTEGRATOR

SIMULATION

RTL ANALYSIS

SYNTHESIS

Netlist

Properties

Project Summary

Device

Tcl Console

Messages

Log

Reports

Design Runs

Power

DRC

Methodology

Timing

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.817 ns	Worst Hold Slack (WHS): 0.033 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3919	Total Number of Endpoints: 3903	Total Number of Endpoints: 2218

All user specified timing constraints are met.

Flow Navigator

PROJECT MANAGER

IP INTEGRATOR

SIMULATION

RTL ANALYSIS

SYNTHESIS

Netlist

Properties

Project Summary

Device

Tcl Console

Messages

Log

Reports

Design Runs

Power

DRC

Methodology

Timing

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.817 ns	Worst Hold Slack (WHS): 0.033 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 3919	Total Number of Endpoints: 3903	Total Number of Endpoints: 2218

All user specified timing constraints are met.

Flow Navigator

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

SIMULATION

- Run Simulation

RTL ANALYSIS

- Open Elaborated Design
- Report Methodology
- Report DRC
- Report Noise
- Schematic

SYNTHESIS

- Run Synthesis
- Open Synthesized Design
 - Constraints Wizard
 - Edit Timing Constraints
 - Set Up Debug
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction

SYNTHESIZED DESIGN - xc7a35tcpg236-1L

Sources

Netlist

- alu
- Nets (31)
- Leaf Cells (23)

Properties

Select an object to see properties

Project Summary

Device

X0Y2 X1Y2

X0Y0 X1Y0

Tcl Console

Messages

Log

Reports

Design Runs

Warning (1)

Info (22)

Status (18)

Show All

Vivado Commands (3 Infos)

- General Messages (3 Infos)
 - [IP_Flow 19-234] Refreshing IP repositories
 - [IP_Flow 19-1704] No user IP repositories specified
 - [IP_Flow 19-2313] Loaded Vivado IP repository D:/programmes/vivado/Vivado/2019.1/data/ip.
- Synthesis (1 warning, 15 Infos)
 - [Common 17-349] Got license for feature 'Synthesis' and/or device 'xc7a35t'
 - [Device 21-403] Loading part xc7a35tcpg236-1L

File Edit Flow Tools Reports Window Layout View Help Q: QUICK ACCESS

Ready

Default Layout

Flow Navigator

ELABORATED DESIGN - xc7a35t0pg230-1L

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Open Elaborated Design
- Report Methodology
- Report DRC
- Report Noise
- Schematic

SYNTHESIS

- Run Synthesis
- Open Synthesized Design

IMPLEMENTATION

- Run Implementation

Sources: Netlist x

plu

- Nets (32)
- Leaf Cells (10)

Properties

Select an object to see properties

Project Summary x Schematic x

9 Cells 12 I/O Ports 32 Nets

Tcl Console Messages Log Reports Design Runs x

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP	Start	Elapsed	Run Strategy
synth_1	constrs_1	Not started															Vivado Synthesis Defaults (Vivado Synthesis 2019)
impl_1	constrs_1	Not started															Vivado Implementation Defaults (Vivado Implementation 2019)

implementation using (Binary, one not, gray)

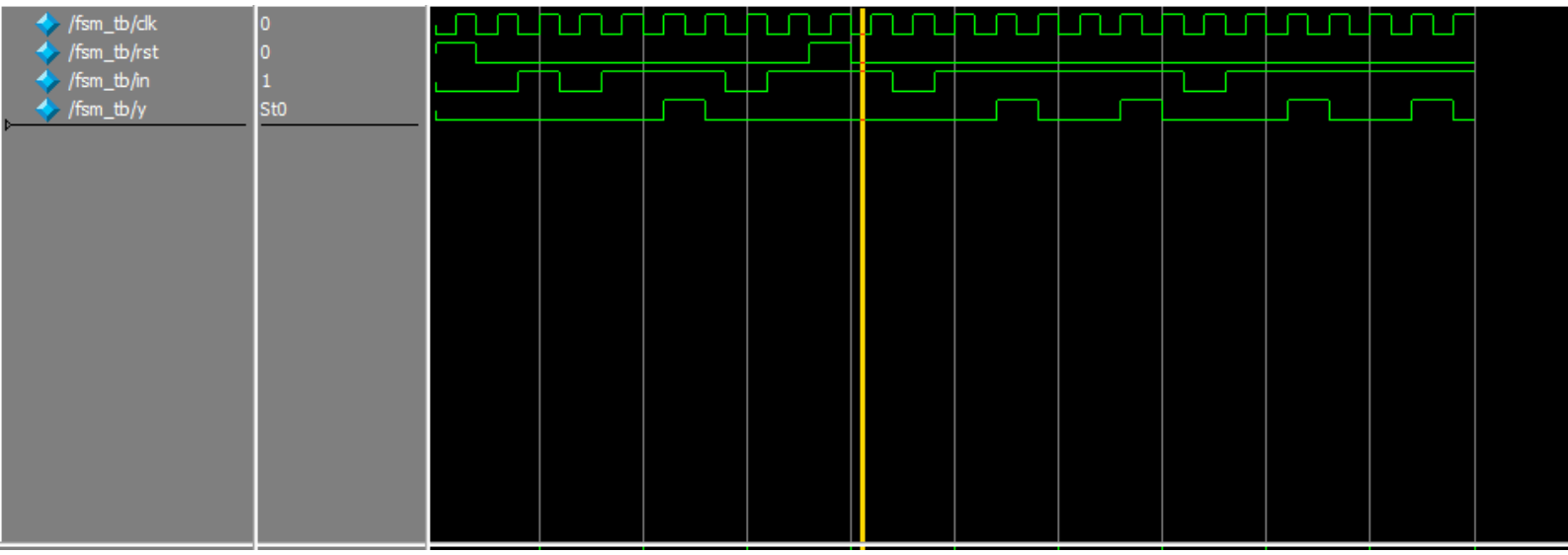
doesn't affect performance But $\begin{cases} \rightarrow \text{area} \\ \rightarrow \text{speed} \\ \rightarrow \text{power} \end{cases}$

Speed: one not > gray > binary

of bits needed: one not > gray = binary

Power Consumption: Binary > gray > one not

State transition: Binary > one not (2) > gray (1)





```
1  module fsm(  
2  
3  input wire clk,rst,  
4  input wire in,  
5  output reg y  
6  );  
7  
8  localparam  A = 2'b00,  
9              B = 2'b01,  
10             C = 2'b10,  
11             D = 2'b11;  
12  
13  (* fsm_encoding = "binary" *)  
14  reg[1:0] cs,ns;  
15  
16  //sequential part  
17  
18  always @(posedge clk or posedge rst)begin  
19      if(rst) cs <= A;  
20      else    cs <= ns;  
21  end  
22  
23  // combintional mealy -> input + statse  
24  always @(in , cs) begin  
25  
26  case(cs)  
27      A: ns = in? B : A ;  
28      B: ns = in? C : B ;  
29      C: ns = in? D : C ;  
30      D: ns = in? B : A ;  
31      default ns = A ;  
32  endcase  
33      y = (cs == D);  
34  end  
35  
36  endmodule  
37
```



```
1  module fsm(  
2  
3  input wire clk,rst,  
4  input wire in,  
5  output reg y  
6  );  
7  
8  localparam  A = 2'b00,  
9              B = 2'b01,  
10             C = 2'b11,  
11             D = 2'b10;  
12  
13  (* fsm_encoding = "gray" *)  
14  reg[1:0] cs,ns;  
15  
16  //sequential part  
17  
18  always @(posedge clk or  posedge rst)begin  
19      if(rst) cs <= A;  
20      else    cs <= ns;  
21  end  
22  
23  // combintional mealy -> input + statse  
24  always @(in , cs) begin  
25  
26  case(cs)  
27      A: ns = in? B : A ;  
28      B: ns = in? C : B ;  
29      C: ns = in? D : C ;  
30      D: ns = in? B : A ;  
31      default ns = A ;  
32  endcase  
33      y = (cs == D);  
34  end  
35  
36  endmodule
```




```
1  module fsm(  
2  
3  input wire clk,rst,  
4  input wire in,  
5  output reg y  
6  );  
7  
8  localparam  A = 2'b0001,  
9              B = 2'b0010,  
10             C = 2'b0100,  
11             D = 2'b1000;  
12  
13  (* fsm_encoding = "onehot" *)  
14  reg[1:0] cs,ns;  
15  
16  //sequential part  
17  
18  always @(posedge clk or posedge rst)begin  
19      if(rst) cs <= A;  
20      else    cs <= ns;  
21  end  
22  
23  // combintional mealy -> input + statse  
24  always @(in , cs) begin  
25  
26  case(cs)  
27      A: ns = in? B : A ;  
28      B: ns = in? C : B ;  
29      C: ns = in? D : C ;  
30      D: ns = in? B : A ;  
31      default ns = A ;  
32  endcase  
33      y = (cs == D);  
34  end  
35  
36  endmodule
```